Closing Manager: what it depends on, and what we need to take over

The month-end close macro works, but still reaches two Capgemini-hosted systems. Executive summary of the dependencies, the risks, and the route to running it ourselves.

THE SHORT ANSWER

Almost — and the gap is smaller than it looks. The macro's real work already runs entirely on the user's PC. What sits outside our control is **configuration, an activity log, and one small executable**. Nothing there requires Capgemini's infrastructure.

2 Capgemini-hosted systems still required at run time

4 settings tables held on their SharePoint, not in our file

0 are technically hard to replace

1 What it does

An Excel VBA macro that runs the month-end close end to end: **drives SAP** via GUI Scripting (SE16 extracts, `ZGLRME`, `ZGE132`, `AA02`, `ZGLGWUL`), exports and re-imports the data, posts the closing entries; **creates the PDFs** through `PDFCreator`; **merges them** with `GiosPSMC.exe` into `<CC> Closure <MM> <YYYY>.pdf`; and **reads/writes configuration** over SOAP to a Capgemini SharePoint site.

SAP is driven **by screen element ID**, not through a BAPI — it is a screen-scraper, so it is inherently sensitive to SAP UI changes, pop-ups and authorisation differences.

2 The two external dependencies

Capgemini SharePoint NOT OURS

Holds the macro's **settings tables** and receives a **log of every close we run**. Read at startup over SOAP, written to during runs. Needs the corporate network or VPN.

```
https://troom-x.capgemini.com/sites/InternationalPaper/r2a/_vti_bin/Lists.asmx
```

Risk: if the site is retired or we are off their network, the macro loads **no settings and reports no error**. Our closing activity is also recorded on a vendor system.

Capgemini Poland file server NOT OURS

A Kraków share holding the PDF merge utility `GiosPSMC.exe`, copied locally on first run.

```
\\pl-krabpo-fsc01\ipa$\R2R\R2R - IP EU GL West\USEFUL\pdf\merger\GiosPSMC.exe
```

Risk: low — a one-time file copy. Once the utility is on the PC the share is never touched again.

The rest is already ours: KEEP **SAP** — legitimate, it is the source of the numbers. DEPLOY **PDFCreator** — desktop install; printer must be named exactly `PDFCreator`. OURS **Local folders** — `C:\pdf\` and `C:\_Files to Transfer\MONTH END CLOSE\`; the workbook must sit in a genuine local folder (running it from OneDrive/SharePoint breaks it).

3 What is actually held on their SharePoint

Not accounting data — configuration plus an activity log. All four would fit in a worksheet or on our own SharePoint.

LIST	CONTENTS	USE
CCCrossList	Per company code: posting block, location-closed flag, CPC allowed/not-allowed under GAAP	Read at startup; maintained from a form in the workbook

LIST	CONTENTS	USE
ClosingVariants	SAP variant name, currency, FC/USD flag per company code	Read at startup
ProfitCenters	Profit-centre reference list	Read at startup; cleared and rewritten
ClosingTracker	User name, company code, period, year, timestamp, success flag	Written on three close failures; read to confirm the period was refreshed

One point to raise with Capgemini

Our close failures are logged to their system. When one of three checks fails (ZGE132 posting, GTB1, ZGE1174) the macro writes the Windows user name, company code, period, timestamp and an error note into `ClosingTracker` on their SharePoint. `UpdateData` also logs every time it runs. Confirm this is agreed, who can access it, and what the retention is.

4 Preflight Check — can users run it on its own?

Yes. It runs standalone, and it changes nothing.

`PreflightCheck` is a self-contained, read-only macro added in V4. It does not start the close, create folders, write files, or drive SAP — it only detects whether a session exists. Safe to run at any time.

How: Alt + F8 → PreflightCheck → Run (it appears in the macro list automatically), or from a button on the `START` sheet.

Reports pass/fail on: workbook location · working drive · SAP session · PDFCreator · PDF merger · company code — turning every dependency above into a check before anyone starts a period close.

5 What we should do

1. **Take a permanent copy of the merge utility**
   Copy `GiosPSMC.exe` to a location we own, or deploy it with the workbook — removes that dependency outright. Confirm with Capgemini what it is and how it is licensed first.
2. **Request an export of the four settings tables**
   Plus who maintains them today and how often they change — this defines exactly what we are taking over.
3. **Decide where the settings live**
   Hidden worksheets in the workbook (removes the server entirely) or a list on our own SharePoint, depending on how many people maintain them and whether we need an audit trail.
4. **Repoint or disable the SharePoint calls**
   V4 isolates the address in one named constant (`CM_SP_BASE`), so redirecting is a one-line change rather than a hunt through seven call sites.
5. **Make failure loud instead of silent**
   Check the configuration actually loaded, so an unreachable endpoint stops the run. **The most important safety item** — worth doing independently of any migration.
6. **Settle ownership and support of the macro**
   The code names an owner and reviewer on a Capgemini automation register. If it is becoming ours, we need the source, the right to modify it, and a named internal owner.

Already delivered. A hardened **V4-CIO** build is available now: fixes the working-drive defect (which silently produced no output on any PC with a D: drive), blocks running from OneDrive, prevents the macro hanging indefinitely, fails clearly when the merge utility or PDFCreator is missing, and adds the Preflight Check. It also gathers both external addresses into named constants — which is what turns steps 1 and 4 into configuration changes rather than a code rewrite.

Prepared by the Continuous Improvement Team from a static review of the workbook's VBA source. No macro was executed and no live system contacted; addresses and list names are those written into the workbook's own code.